Министерство образования Республики Беларусь

Учреждение образования

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина Архитектура вычислительных систем

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему

УМНОЕ ЗЕРКАЛО С ПЕРСОНАЛЬНЫМИ РЕКОМЕНДАЦИЯМИ ДЛЯ ПОЛЬЗОВАТЕЛЯНА БАЗЕ ESP32

БГУИР КП 6-05-0612-02 075 ПЗ

Студент	А. Н. Михалко
Руководитель	А. Н. Марков
Нормоконтроллёр	А. А. Калиновская

Минск 2026

СОДЕРЖАНИЕ

Введение 5
1 Архитектура вычислительной системы 7
1.1 Структура и архитектура вычислительной системы 7
1.2 История, версии и достоинства ESP32 7
1.3 Обоснование выбора вычислительной системы 8
1.4 Анализ выбранной вычислительной системы для написания программы 9
2 Платформа программного обеспечения 10
2.1 Структура и архитектура платформы 10
2.2 История, версии и достоинства платформы 10
2.3 Обоснование выбора платформы 10
2.4 Анализ операционной системы для написания программы 10
3 Теоретическое обоснование разработки программного продукта 12
3.1 Обоснование необходимости разработки 12
3.2 Технологии программирования, используемые для решения задач . . . 12
3.3 Связь архитектуры вычислительной системы с разрабатываемым программным обеспечением 12
4 Проектирование функциональных возможностей программы 14
4.1 Обоснование и описание функций программного обеспечения 14
4.2 Ограничения и требования к программе 14
5 Архитектура разрабатываемой программы 16
5.1 Общая структура программы 16
5.2 Описание функциональной схемы программы 16
5.3 Описание блок-схемы алгоритма программы 16
Заключение 19
Список использованных источников 20

ВВЕДЕНИЕ

Развитие интернета вещей и рост доступности беспроводных микроконтроллеров привели к появлению «умных» устройств, которые не только отображают данные, но и анализируют состояние человека. Умное зеркало является характерным примером такого устройства: пользователь подходит к нему ежедневно, поэтому оно удобно для ненавязчивого наблюдения за состоянием кожи лица и глаз. Интерес к дистанционному контролю самочувствия растёт вместе с распространением телемедицинских сервисов, однако готовых недорогих устройств, которые выполняют такую оценку без участия пользователя, немного. Современные системы на кристалле с Wi-Fi и поддержкой камер справляются со сбором данных, а ресурсоёмкие вычисления, такие как работа нейронных сетей и генерация текста, целесообразно выполнять на сервере. Проект находится на стыке архитектуры вычислительных систем, встраиваемого программирования и машинного обучения.

Серверная часть умного зеркала уже разработана. Это REST-сервис на языке Python с использованием фреймворка Flask [1], который находит на изображении лицо и глаза с помощью каскадов Хаара, определяет состояние кожи лица (шесть классов) и глаз (три класса) с помощью свёрточной нейронной сети MobileNetV2 [2] и формирует для пользователя текстовые рекомендации с помощью локальной языковой модели Llama3, развёрнутой в контейнере Docker [3]. Однако серверу необходим аппаратный клиент, то есть устройство, которое определяет момент съёмки, получает снимок, отправляет его на сервер и показывает ответ пользователю. Создание такого клиента на базе микроконтроллеров ESP32 является предметом курсового проекта по дисциплине «Архитектура вычислительных систем». Основное внимание уделяется выбору аппаратной платформы и тому, как её особенности определяют структуру программы.

Целью работы является разработка программного обеспечения умного зеркала на языке C/C++ в среде VS Code с использованием PlatformIO. Для достижения цели необходимо решить следующие задачи:

– обосновать выбор микроконтроллеров, датчиков и дисплея, составляющих аппаратную часть;

– выбрать программную платформу и операционную систему;

– спроектировать функции и архитектуру программы клиента;

– реализовать программу, которая не допускает зависаний при ожидании сети и самостоятельно восстанавливает соединение с Wi-Fi.

Детальная постановка задачи выглядит следующим образом. Физически система состоит из сервера и двух модулей с Wi-Fi. Главный контроллер ESP32 DevKit V1 опрашивает два датчика расстояния: ультразвуковой HC-SR04 и лазерный VL53L0X. Когда оба датчика показывают, что пользователь находится на расстоянии, пригодном для съёмки лица, контроллер запрашивает снимок у модуля ESP32-CAM, на котором установлена камера OV2640. Полученный кадр в формате JPEG контроллер отправляет на сервер HTTP-запросом, дожидается JSON-ответа и выводит результат на TFT-дисплей. Два светодиода,

красный и зелёный, показывают состояние устройства и ошибки. Программа не должна блокироваться при ожидании сети и должна экономно расходовать оперативную память. Следует подчеркнуть, что зеркало даёт рекомендации информационного характера и не ставит медицинских диагнозов. Общая схема системы показана на рисунке 0.1.

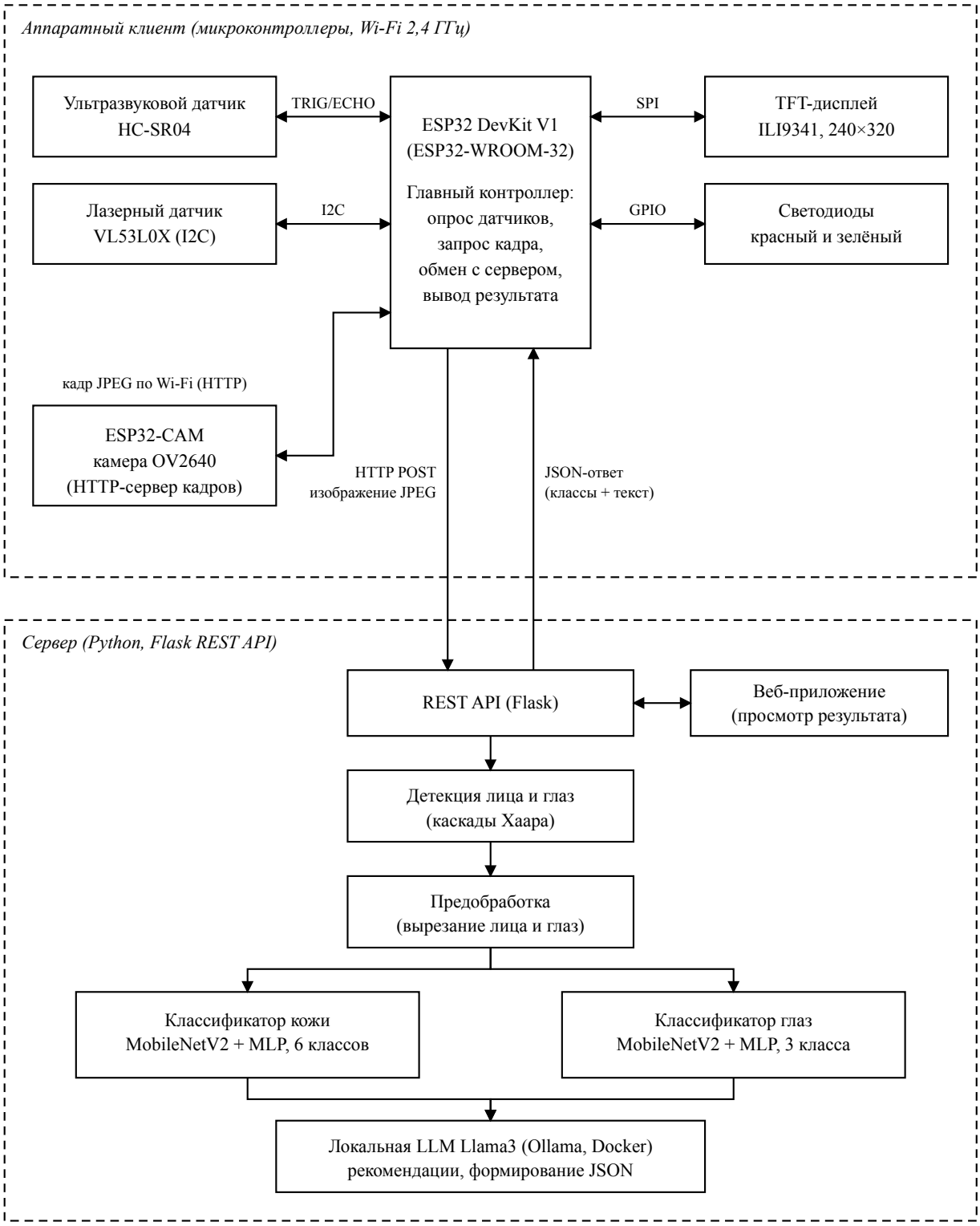


Рисунок 0.1 – Архитектура системы умного зеркала

1 АРХИТЕКТУРА ВЫЧИСЛИТЕЛЬНОЙ СИСТЕМЫ

1.1 Структура и архитектура вычислительной системы

Система умного зеркала разделена на два уровня: микроконтроллеры отвечают за сбор данных и взаимодействие с пользователем, а сервер выполняет нейросетевую обработку. Такое разделение обосновано тем, что микроконтроллеры дёшевы, потребляют мало энергии и напрямую работают с датчиками, камерой и дисплеем, а сервер обеспечивает необходимую вычислительную мощность.

Микроконтроллерный уровень состоит из двух узлов на базе систем на кристалле семейства ESP32. Базовый ESP32 объединяет процессор, память, модули Wi-Fi и Bluetooth и набор периферийных интерфейсов [4]. Вычислительную часть образуют одно или два 32-разрядных ядра Xtensa LX6 с частотой до 240 МГц. В двухъядерном варианте ядра работают с общей памятью, при этом по умолчанию ядро PRO_CPU обслуживает сетевой стек Wi-Fi, а ядро APP_CPU выполняет задачи приложения [5]. Объём встроенной SRAM невелик (520 КБ), поэтому при необходимости подключается внешняя память PSRAM [4]. Аппаратные ускорители криптографических алгоритмов снижают нагрузку на процессор при защищённой передаче данных.

Первый узел, ESP32 DevKit V1 на модуле ESP32-WROOM-32, является главным контроллером. К нему подключены датчики расстояния, дисплей и светодиоды. Второй узел, ESP32-CAM, выполняет единственную функцию: по запросу делает снимок и отдаёт его по сети. Периферийные устройства подключаются к главному контроллеру по следующим интерфейсам:

– ультразвуковой датчик HC-SR04 работает через два выхода общего назначения: импульс запуска и измерение длительности ответного импульса [6];

– лазерный датчик VL53L0X подключён по шине I2C и передаёт расстояние в миллиметрах [7];

– дисплей ILI9341 с разрешением 240×320 точек управляется по шине SPI [8];

– светодиоды подключены к выходам общего назначения через токоограничивающие резисторы.

Датчик HC-SR04 питается напряжением 5 В, а его выход ответного импульса выдаёт сигнал такого же уровня. Выводы ESP32 рассчитаны на 3,3 В, поэтому выход датчика подключается через резисторный делитель напряжения на 1 кОм и 2 кОм, который понижает сигнал примерно до 3,3 В. Для сборки без пайки используется макетная плата, а питание всей системы обеспечивает блок питания 5 В 2 А.

1.2 История, версии и достоинства ESP32

Компания Espressif выпустила ESP32 в 2016 году как преемника одноядерного ESP8266 [9]. Впоследствии семейство расширилось: появились

одноядерный ESP32-S2 без Bluetooth, ESP32-C3 на архитектуре RISC-V и двухъядерный ESP32-S3 [9]. Базовый ESP32 остаётся самой распространённой серией, для неё существует наибольшее число отладочных плат, библиотек и примеров.

Основным достоинством ESP32 является высокая степень интеграции: процессор, модуль Wi-Fi и периферия находятся на одном недорогом кристалле, для работы которого достаточно добавить питание [4]. Два ядра позволяют разделить обработку сетевых протоколов и логику приложения, а обширная экосистема предоставляет множество библиотек и недорогих отладочных плат. К недостаткам относятся работа Wi-Fi только в диапазоне 2,4 ГГц и небольшой объём оперативной памяти, недостаточный для запуска таких операционных систем, как Linux. Поэтому перенос машинного обучения на сервер является необходимостью.

1.3 Обоснование выбора вычислительной системы

Устройству требуются Wi-Fi, поддержка камеры, дисплея и датчиков, а также достаточный объём памяти для буфера кадра. Одноплатные компьютеры с Linux, например Raspberry Pi, для этой задачи избыточны и дороже, так как все тяжёлые вычисления вынесены на сервер.

Для камерного узла выбран недорогой и распространённый модуль ESP32-CAM с сенсором OV2640. Драйвер esp32-camera поддерживает его без дополнительных доработок [10]. Модуль оснащён внешней памятью PSRAM, что важно для хранения кадров. Использование отдельной платы для камеры вместо подключения камеры к главному контроллеру объясняется тем, что ESP32-CAM занимает почти все свои выводы собственной камерой и не оставляет их для дисплея и датчиков, а на DevKit V1 выводы камеры не выведены. Кроме того, разделение узлов позволяет разместить камеру в удобном месте зеркала независимо от контроллера.

Главным контроллером выбран ESP32 DevKit V1. Число его выводов достаточно для подключения двух датчиков, дисплея по SPI и светодиодов, а два ядра позволяют обновлять изображение на дисплее, не нарушая работу с сетью. Плата не содержит внешней PSRAM, поэтому принимаемый от камеры кадр в формате JPEG должен помещаться во внутреннюю оперативную память. Для этого камеру настраивают на умеренное разрешение и сжатие, при которых размер кадра составляет десятки килобайт.

Использование двух разных по принципу действия датчиков расстояния повышает надёжность определения присутствия пользователя. Ультразвуковой датчик имеет широкий конус излучения и может реагировать на посторонние предметы рядом с пользователем. Лазерный датчик имеет узкий луч и точнее измеряет расстояние до лица. Съёмка запускается только тогда, когда показания обоих датчиков совпадают с допустимой погрешностью, что снижает число ложных срабатываний.

Вынос нейронных сетей на сервер обоснован их вычислительной сложностью: для обработки одного изображения MobileNetV2 требует порядка 300

млн операций умножения со сложением [2]. Микроконтроллер справится с этим только в упрощённом (квантованном) виде и с низкой скоростью. Кроме того, серверная часть уже разработана, поэтому за зеркалом целесообразно оставить сбор данных и вывод результатов.

1.4 Анализ выбранной вычислительной системы для написания программы

Особенности аппаратной части напрямую определяют приёмы программирования. Встроенной оперативной памяти немного, поэтому кадр с камеры целесообразно получать уже сжатым в формате JPEG, а не в виде несжатого изображения. Это разгружает процессор, ускоряет передачу по Wi-Fi и позволяет разместить кадр на главном контроллере без внешней памяти [10].

Драйвер камеры получает данные с использованием прямого доступа к памяти (DMA) [10]. На камерном узле задачу обслуживания камеры целесообразно закрепить за одним ядром, чтобы она не мешала сетевым задачам на другом. На главном контроллере используется та же схема: сетевые задачи выполняются на ядре PRO_CPU, а опрос датчиков и обновление дисплея на ядре APP_CPU [5].

Измерения датчиков имеют разную природу. Ультразвуковой датчик требует точного измерения длительности импульса порядка десятков и сотен микросекунд, а лазерный датчик передаёт готовое значение по шине I2C. Поэтому опрос датчиков выносится в отдельную задачу с фиксированным периодом, а результат усредняется по нескольким измерениям для подавления шума.

Wi-Fi в диапазоне 2,4 ГГц может работать нестабильно, поэтому программа должна задавать тайм-ауты на все сетевые операции и автоматически восстанавливать соединение при обрыве без перезагрузки устройства. Для защиты от зависаний используются аппаратные сторожевые таймеры, которые перезагружают кристалл, если программа перестала отвечать [4].

Таким образом, программа строится на основе событий: отдельные задачи обмениваются данными через очереди, передавая указатели на буферы, а не копируя их содержимое.

2 ПЛАТФОРМА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1 Структура и архитектура платформы

Программа разрабатывается в среде ESP-IDF, официальном фреймворке компании Espressif [9]. В соответствии с заданием используются язык C/C++, редактор VS Code и инструмент PlatformIO, в настройках которого указывается `framework = espidf` [11].

Архитектура платформы многоуровневая. Нижний уровень образуют загрузчики прошивки. Выше запускается планировщик FreeRTOS, который вызывает функцию `app_main`, после чего работа приложения строится как набор параллельных задач [9]. Следующий уровень составляют драйверы ESP-IDF (Wi-Fi, HTTP-клиент, шины I2C и SPI, управление выводами) и подключаемые компоненты, например драйвер камеры [10]. Сборка выполняется с помощью CMake, а общие настройки хранятся в файле `sdkconfig` [11]. В ESP-IDF используется собственная версия FreeRTOS, распределяющая задачи между двумя ядрами микроконтроллера [5].

2.2 История, версии и достоинства платформы

ESP-IDF разрабатывается открыто на GitHub. Каждая стабильная ветка поддерживается ограниченное время, а при переходе между крупными версиями возможны изменения, нарушающие совместимость [12]. По этой причине в PlatformIO версию платформы фиксируют явно, чтобы проект не перестал собираться после очередного обновления [11].

Основными достоинствами ESP-IDF являются полный доступ к функциям кристалла (работа с двумя ядрами, тонкая настройка сети, аппаратная криптография) и подробная документация. Недостатками являются большая, чем в Arduino, сложность написания кода и более длительная компиляция.

2.3 Обоснование выбора платформы

В PlatformIO для ESP32 доступны два фреймворка: Arduino и ESP-IDF [13]. Arduino упрощает разработку, но скрывает важные настройки: управление сетевыми буферами, тайм-аутами и распределением задач по ядрам. ESP-IDF обеспечивает полный контроль над аппаратными ресурсами. Кроме того, разработка на ESP-IDF лучше соответствует дисциплине «Архитектура вычислительных систем», так как наглядно показывает, как программное обеспечение управляет ресурсами кристалла.

2.4 Анализ операционной системы для написания программы

Основой программы является операционная система реального времени FreeRTOS. Она использует вытесняющую многозадачность, при которой более

приоритетная задача прерывает менее приоритетную, а любую задачу можно привязать к определённому ядру [5].

Для координации задач FreeRTOS предоставляет очереди, мьютексы и группы событий. Очереди применяются для передачи указателей на кадры от задачи получения снимка к задаче отправки по Wi-Fi. Мьютексы не позволяют двум задачам одновременно обновлять дисплей. Группы событий вместе с ожиданием по тайм-ауту позволяют реализовать ожидание подключения к Wi-Fi и ответа сервера без циклов активного опроса, загружающих процессор [9].

Задачи опроса датчиков, работы с сетью и интерфейса выполняются независимо и взаимодействуют через средства FreeRTOS, что обеспечивает отзывчивость устройства и исключает зависания при ожидании ответа сервера.

3 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРОГРАММНОГО ПРОДУКТА

3.1 Обоснование необходимости разработки

Готовая серверная часть принимает изображение по протоколу HTTP и возвращает результат анализа в формате JSON. Однако без аппаратного клиента пользователь вынужден вручную фотографировать себя и загружать снимок через веб-приложение. Это неудобно для ежедневного использования. Программа для микроконтроллеров автоматизирует весь процесс: определяет, что пользователь находится перед зеркалом, получает снимок без участия человека и показывает результат на дисплее. Готовых прошивок под выбранный набор оборудования и протокол сервера не существует, поэтому программу необходимо разработать.

3.2 Технологии программирования, используемые для решения задач

Для решения поставленных задач используются следующие технологии:

– язык C/C++ и фреймворк ESP-IDF, обеспечивающие доступ к ресурсам кристалла;

– FreeRTOS для организации параллельных задач, очередей и таймаутов;

– HTTP-клиент ESP-IDF для получения кадра от камерного узла и отправки его на сервер методом POST;

– драйвер esp32-camera для работы с сенсором OV2640 на камерном узле [10];

– драйверы шин I2C и SPI и компонент `esp_lcd` для работы с датчиком VL53L0X и дисплеем ILI9341 [9];

– разбор JSON-ответа сервера библиотекой cJSON, входящей в состав ESP-IDF.

Сетевое взаимодействие построено по схеме «клиент – сервер». Главный контроллер выступает клиентом одновременно для двух узлов: от камерного узла он получает кадр запросом GET, а на сервер отправляет его запросом POST. Такой подход не требует прокладки дополнительных проводов между платами, так как обе платы уже подключены к одной сети Wi-Fi.

3.3 Связь архитектуры вычислительной системы с разрабатываемым программным обеспечением

Архитектура ESP32 определяет следующие решения в программе:

– двухъядерная структура позволяет закрепить сетевые задачи за одним ядром, а опрос датчиков и обновление дисплея за другим;

– небольшой объём SRAM на главном контроллере требует хранить кадр в одном буфере и передавать между задачами указатель на него;

– отсутствие у DevKit V1 внешней PSRAM ограничивает размер кадра, поэтому камера настраивается на формат JPEG умеренного разрешения;

– напряжение 3,3 В на выводах ESP32 требует согласования уровней для датчика HC-SR04;

– работа Wi-Fi только в диапазоне 2,4 ГГц делает необходимыми тайм-ауты и автоматическое переподключение;

– наличие аппаратных сторожевых таймеров позволяет перезагрузить устройство, если программа всё же перестала отвечать.

4 ПРОЕКТИРОВАНИЕ ФУНКЦИОНАЛЬНЫХ ВОЗМОЖНОСТЕЙ ПРОГРАММЫ

4.1 Обоснование и описание функций программного обеспечения

Функции программы определяются назначением умного зеркала и свойствами аппаратной части. Программа главного контроллера выполняет следующие функции:

– инициализация периферии: настройка выводов, шины I2C, шины SPI, дисплея и светодиодов;

– подключение к сети Wi-Fi и автоматическое восстановление соединения при обрыве;

– определение присутствия пользователя: измерение расстояния двумя датчиками и принятие решения о начале съёмки, если оба датчика показывают значение в заданном диапазоне в течение заданного времени;

– получение кадра от камерного узла запросом `GET /capture` с таймаутом;

– отправка кадра на сервер запросом `POST` и получение JSON-ответа;

– разбор ответа и вывод на дисплей классов состояния кожи и глаз, а также краткого текста рекомендации;

– индикация состояния светодиодами: зелёный означает успешный результат, красный означает ошибку датчика, камеры, сети или сервера;

– защита от зависаний с помощью сторожевого таймера.

Программа камерного узла выполняет две функции: инициализирует камеру и по запросу GET возвращает один кадр в формате JPEG.

Диапазон расстояний для съёмки принят равным 30–60 см. Это связано с тем, что у сенсора OV2640 фиксированный фокус, а лицо должно занимать значительную часть кадра, чтобы каскады Хаара на сервере уверенно его обнаружили. Требование к времени удержания, равное 1 с, исключает срабатывания при случайном проходе мимо зеркала.

4.2 Ограничения и требования к программе

К программе предъявляются следующие требования:

– ни одна сетевая операция не должна блокировать работу программы на неопределённое время;

– после обрыва Wi-Fi устройство должно восстановить соединение без перезагрузки;

– при ошибке на любом этапе устройство сообщает о ней пользователю и возвращается к ожиданию, а не останавливается;

– выводимый результат сопровождается пояснением, что рекомендации носят информационный характер и не являются медицинским диагнозом.

Отдельное ограничение связано с выводом текста. Рекомендации формирует языковая модель, и их объём может превышать размер экрана 240×320 точек. Поэтому на дисплей выводится сокращённый текст, а полный ответ

доступен в веб-приложении сервера. Для вывода русского текста необходим растровый шрифт, содержащий кириллические символы.

5 АРХИТЕКТУРА РАЗРАБАТЫВАЕМОЙ ПРОГРАММЫ

5.1 Общая структура программы

Программа главного контроллера состоит из нескольких независимых задач FreeRTOS, каждая из которых отвечает за одну функцию:

– задача датчиков периодически опрашивает HC-SR04 и VL53L0X, усредняет измерения и при выполнении условия присутствия отправляет событие в очередь;

– задача связи по событию запрашивает кадр у ESP32-CAM, отправляет его на сервер и получает ответ;

– задача интерфейса выводит на дисплей состояние и результат и управляет светодиодами;

– задача Wi-Fi следит за соединением и восстанавливает его при обрыве.

Задачи обмениваются данными через очереди FreeRTOS: между задачей датчиков и задачей связи передаётся событие присутствия, между задачей связи и задачей интерфейса передаются результат или код ошибки. Кадр хранится в единственном буфере, а между задачами передаётся только указатель на него. Доступ к дисплею защищён мьютексом.

5.2 Описание функциональной схемы программы

Функциональная схема программы совпадает со схемой системы, показанной на рисунке 0.1. Данные движутся следующим образом. Датчики расстояния передают измерения в задачу датчиков, которая принимает решение о начале съёмки. Получив событие, задача связи обращается к камерному узлу по Wi-Fi и получает снимок. Затем снимок передаётся на сервер, где последовательно выполняются детекция лица и глаз, предобработка, классификация состояния кожи и глаз и генерация текстовой рекомендации. JSON-ответ возвращается на главный контроллер, и задача интерфейса выводит его на дисплей и включает зелёный светодиод. Если на любом этапе происходит ошибка, включается красный светодиод и на дисплей выводится сообщение.

5.3 Описание блок-схемы алгоритма программы

Блок-схема алгоритма работы главного контроллера приведена на рисунке 5.1.

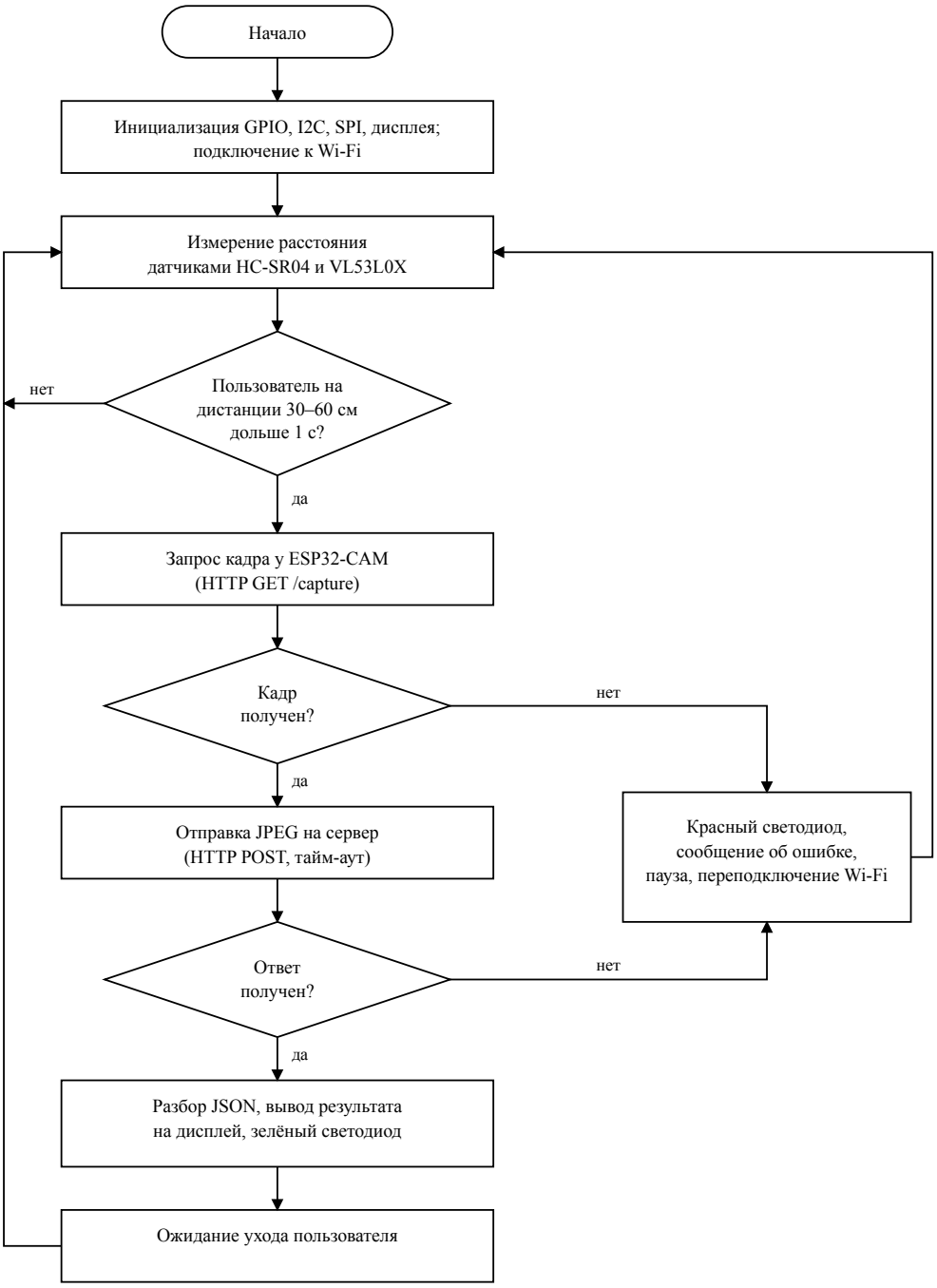


Рисунок 5.1 – Блок-схема алгоритма работы главного контроллера

После включения питания выполняется инициализация выводов, шин I2C и SPI, дисплея и светодиодов, затем устройство подключается к сети Wi-Fi. Далее в цикле измеряется расстояние двумя датчиками. Если пользователь не находится в диапазоне 30–60 см не менее 1 с, измерения продолжаются. В противном случае контроллер запрашивает кадр у ESP32-CAM. Если кадр не получен в течение тайм-аута, включается красный светодиод, на дисплей выводится сообщение об ошибке, выдерживается пауза и при необходимости выполняется переподключение к Wi-Fi, после чего алгоритм возвращается к измерению расстояния. При успешном получении кадра он отправляется

на сервер. Если ответ не получен в течение тайм-аута, выполняется та же обработка ошибки. Полученный JSON-ответ разбирается, результат выводится на дисплей, включается зелёный светодиод. Затем контроллер ожидает, пока пользователь отойдёт от зеркала, чтобы не отправлять повторные запросы для одного и того же человека, и возвращается к измерению расстояния.

ЗАКЛЮЧЕНИЕ

Текст заключения будет написан после реализации и тестирования программы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Pallets. Flask Documentation [Электронный ресурс] / Pallets. – Режим доступа: https://flask.palletsprojects.com/. – Дата доступа: 25.10.2026.

[2] Sandler, M. MobileNetV2: Inverted Residuals and Linear Bottlenecks / M. Sandler [и др.] // Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). – 2018.. – MobileNetV2: Inverted Residuals and Linear Bottlenecks. – С. 4510-4520.

[3] Ollama Documentation [Электронный ресурс]. – Режим доступа: https://docs.ollama.com/. – Дата доступа: 22.02.2026.

[4] Espressif Systems. ESP32 Series Datasheet [Электронный ресурс] / Espressif Systems. – Режим доступа: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf. – Дата доступа: 28.09.2026.

[5] Espressif Systems. FreeRTOS (ESP-IDF): dual-core SMP implementation [Электронный ресурс] / Espressif Systems. – Режим доступа: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/system/freertos_idf.html. – Дата доступа: 28.09.2026.

[6] HC-SR04 Ultrasonic Sensor: Datasheet [Электронный ресурс]. – Режим доступа: https://cdn.sparkfun.com/datasheets/Sensors/Proximity/HCSR04.pdf. – Дата доступа: 25.10.2026.

[7] STMicroelectronics. VL53L0X: Time-of-Flight Ranging Sensor. Datasheet [Электронный ресурс] / STMicroelectronics. – Режим доступа: https://www.st.com/resource/en/datasheet/vl53l0x.pdf. – Дата доступа: 25.10.2026.

[8] ILI Technology Corp. ILI9341: 240x320 Active Matrix TFT LCD Single Chip Driver. Datasheet [Электронный ресурс] / ILI Technology Corp.. – Режим доступа: https://cdn-shop.adafruit.com/datasheets/ILI9341.pdf. – Дата доступа: 25.10.2026.

[9] Espressif Systems. ESP-IDF Programming Guide [Электронный ресурс] / Espressif Systems. – Режим доступа: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/index.html. – Дата доступа: 28.09.2026.

[10] Espressif Systems. ESP32 Camera Driver (esp32-camera) [Электронный ресурс] / Espressif Systems. – Режим доступа: https://github.com/espressif/esp32-camera. – Дата доступа: 28.09.2026.

[11] PlatformIO Labs. Espressif IoT Development Framework: PlatformIO documentation [Электронный ресурс] / PlatformIO Labs. – Режим доступа: https://docs.platformio.org/en/latest/frameworks/espidf.html. – Дата доступа: 28.09.2026.

[12] Espressif Systems. ESP-IDF Versions [Электронный ресурс] / Espressif Systems. – Режим доступа: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/versions.html. – Дата доступа: 28.09.2026.

[13] PlatformIO Labs. Espressif 32: development platform, PlatformIO documentation [Электронный ресурс] / PlatformIO Labs. – Режим доступа: https://docs.platformio.org/en/latest/platforms/espressif32.html. – Дата доступа: 28.09.2026.

[14] Espressif Systems. ESP32-S3 Series Datasheet [Электронный ресурс] / Espressif Systems. – Режим доступа: https://www.espressif.com/sites/default/files/documentation/esp32-s3_datasheet_en.pdf. – Дата доступа: 28.09.2026.

[15] Espressif Systems. Releases: espressif/esp-idf [Электронный ресурс] / Espressif Systems. – Режим доступа: https://github.com/espressif/esp-idf/releases. – Дата доступа: 28.09.2026.